

PHP

SOMMAIRE

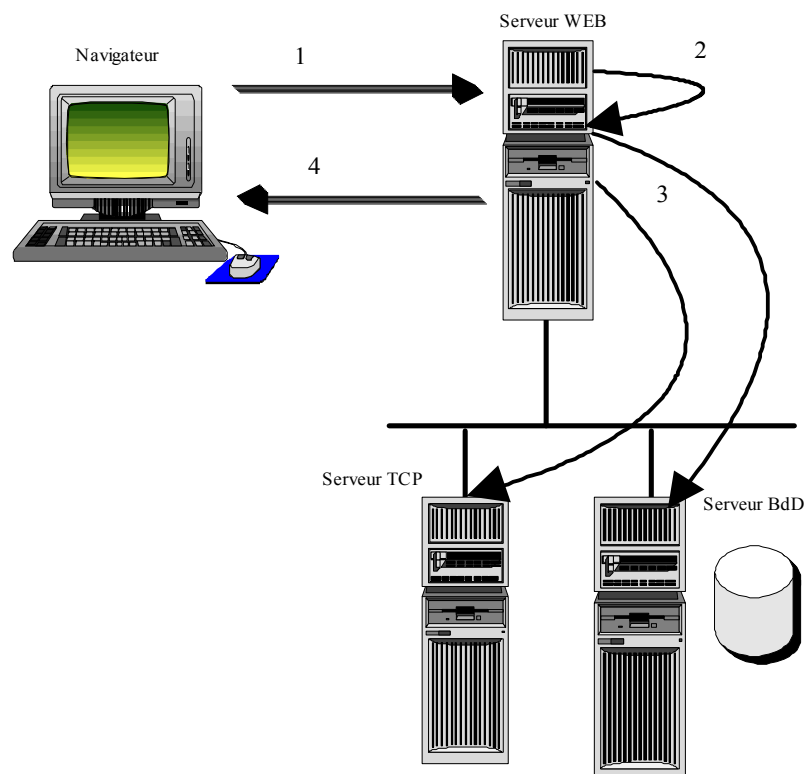
1	QU'EST CE QUE PHP ?	3
2	BREF HISTORIQUE	4
3	PROGRAMMATION PHP	4
3-1	L'AIDE	4
3-2	LES COMMENTAIRES	4
3-3	L'AFFICHAGE	4
3-4	LES VARIABLES	5
3-5	LES CONSTANTES	6
3-6	LES TYPES DE DONNÉES.....	6
4	LES OPÉRATEURS.....	8
5	LES STRUCTURES DE CONTRÔLE	9
6	FONCTIONS ET CLASSES.....	11
6-1	LES FONCTIONS	11
•	FONCTIONS VARIABLES	11
•	VALEUR D'UN PARAMÈTRE PAR DÉFAUT	12
•	PASSAGE DE PARAMÈTRE PAR RÉFÉRENCE	12
•	NOMBRE VARIABLE DE PARAMÈTRES	12
•	VISIBILITÉ DES VARIABLES.....	12
•	VARIABLE STATIC.....	12
6-2	LES CLASSES EN PHP 5	12
•	CONSTRUCTEUR	12
•	DESTRUCTEUR.....	12
•	VISIBILITÉ	12
•	ACCESSEURS	13
•	__toString	13
•	HÉRITAGE.....	13
•	STATIC	13
•	CONSTANTE DE CLASSE.....	13
•	MÉTHODES ET CLASSES ABSTRAITES	13
•	INTERFACES	13
•	FINAL	13
•	STATIC	13
•	INSTANCEOF.....	13
•	CLONE.....	14
•	__autoload	14
•	SPL - PARCOURS D'OBJET.....	14
•	TYPAGE OBJET.....	14
•	EXCEPTIONS	14
7	GESTION DES FORMULAIRES.....	15
7-1	RAPPELS.....	15
7-2	RÔLE DE PHP DANS LES FORMULAIRES – RÉCUPÉRATION DES DONNÉES	15
7-3	QUELQUES CONTRÔLES ET LA RÉCUPÉRATION DES DONNÉES	16

• ZONES CONTENANT DU TEXTE :	16
• GROUPE DE BOUTONS RADIO :	16
• CASES À COCHER :	16
• LISTE À SÉLECTION UNIQUE :	17
• LISTE À SÉLECTION MULTIPLE :	17
• BOUTON DE VALIDATION :	17
• BOUTON IMAGE :	18
7-4 PROBLÈMES AVEC LES DONNÉES SAISIES	18
• MAGIC_QUOTES :	18
• INTERPRÉTATIONS DU NAVIGATEUR :	19
• RENVOYER SUR UNE AUTRE PAGE	20
8 IDENTIFICATION - PASSAGE DE DONNÉES ENTRE PAGES	20
8-1 IDENTIFICATION D'UN UTILISATEUR	21
8-2 IDENTIFIANT UNIQUE	21
8-3 PASSAGE DE DONNÉES ENTRE PAGES	21
• PAR L'URL	21
• PAR UNE ZONE DE FORMULAIRE CACHÉE	22
• PAR LES COOKIES	22
9 GESTION DES SESSIONS	23
9-1 PRÉSENTATION	23
9-2 FONCTIONS PHP GÉRANT LES SESSIONS	23
9-3 GESTION PAR URL OU PAR COOKIES	24
9-4 AUTRES DIRECTIVES	26
10 ACCÈS AUX BASES DE DONNÉES	27
10-1 CONNEXION À UNE BASE DE DONNÉES	27
10-2 DÉCONNEXION D'UNE BASE DE DONNÉES	28
10-3 CHOIX DE LA BASE DE DONNÉES	28
10-4 EXÉCUTION D'UNE REQUÊTE	28
10-5 EXTRACTION DES DONNÉES – NOTION DE "FETCH"	29
• SERVEUR MYSQL	29
• SERVEUR POSTGRES SQL	30
• SERVEUR MS SQL	30
10-6 GESTION DES ERREURS	31
10-7 MAGIC_QUOTES	31
11 GÉRER DES FICHIERS	32
11-1 FICHIERS LOCAUX	32
11-2 UPLOAD	32
11-3 DOWNLOAD	33
12 GÉRER LES SOCKETS	34

PHP

1 Qu'est ce que PHP ?

PHP est un langage de script interprété du côté serveur. Lorsqu'un client utilise un navigateur, il envoie une requête HTTP à un serveur WEB (étape 1). Ce serveur va lire le fichier demandé. Dans ce fichier, il va trouver du code HTML qu'il va laisser intact et du code PHP que le moteur PHP (parser) va interpréter (étape 2 et 3). Le résultat de cette interprétation peut être du code HTML qui sera ajouté au code déjà trouvé. Lorsque tous les scripts PHP auront été interprétés, le serveur WEB va envoyer la page au navigateur du client (étape 4). Dans le fichier demandé, le serveur WEB peut aussi trouver des scripts client (javascript par exemple) dans ce cas, ces scripts sont ajoutés au code HTML et seront exécutés sur la machine client. Les scripts PHP stockés sur le serveurWEB ne sont donc jamais visibles directement par le client puisqu'ils sont exécutés sur ce serveurWEB. Pour qu'un fichier puisse être interprété par le serveurWEB, il doit avoir l'extension php ou php3 ou php4 et non html. Un script PHP est délimité par les balises : `<?php` et `?>` ou `<script language="php">` et `</script>` ou `<?>` et `?>` ou encore `<%php` et `%>` (seules les deux premières seront reconnues quelle que soit la configuration et si on veut inclure le code PHP dans un fichier XML ou XHTML, seule la première sera conforme).



Le langage PHP permet d'accéder à des fichiers (étape 2), à des bases de données (étape 3) ou à un serveur TCP (utilisation de socket étape 3). Dans ce dernier cas, on pourra, à partir d'un simple navigateur, demander des informations à un système technique relié à ce serveur TCP ou agir de ce système technique toujours par ce serveur TCP. La base de données peut servir par exemple à enregistrer des informations sur ce système technique ou à autoriser (ou non) des clients à communiquer avec le système technique. PHP supporte de nombreux protocoles (LDAP, IMAP, SNMP, NNTP, POP3, HTTP ...) et aussi COM et CORBA.

2 Bref historique

Le langage PHP a été conçu en 1994 par Rasmus Lerdorf pour ses besoins personnels avant d'être rendu public au début de l'année 1995. PHP signifiait Personnel Home Page.

En 1997, le projet devient un travail d'équipe et l'interpréteur est réécrit par Zeev Suraski et Andi Gutmans pour donner la version PHP3, version qui s'est rapidement imposée. Elle devient PHP : *Hypertext PreProcessor*.

En 1998, Andi Gutmans et Zeev Suraski commencèrent la réécriture du moteur interne de PHP. L'objectif était d'améliorer les performances de PHP avec les applications complexes, et d'améliorer la modularité du code. Le nouveau moteur, appelé 'Zend Engine' (combinaison des noms de Zeev et Andi), atteint ces objectifs avec succès, et la première version fut publiée vers la moitié de l'année 1999.

La version suivante : PHP4 s'appuie sur le moteur Zend (société privée créée par Suraski et Gutmans) et fut publiée officiellement en mai 2000. PHP4 s'avère plus rapide, plus fiable et plus complet, supporte de nombreux serveurs web (Apache et IIS entre autres) et introduit les sessions HTTP. Les scripts sont désormais compilés puis exécutés.

La dernière version, apparue en 2005, est PHP5. Elle inclut la nouvelle version du Zend Engine 2.0 et intègre de vraies capacités objets : l'encapsulation est respectée ! Les exceptions sont gérées. L'extension MySQL classique est toujours disponible mais maintenant, il faut l'activer explicitement. Une nouvelle extension MySQLi, activée par défaut, a été développée de zéro et adopte une syntaxe mixte orientée objet ou procédurale.

3 Programmation PHP

En PHP, les instructions se terminent par le caractère point virgule. La balise de fin (?>) implique la fin d'une instruction, et donc ajoute implicitement un point virgule. Il existe une grande similitude avec le langage C. Seules les différences seront développées.

3-1 l'aide

Il est possible d'obtenir de "l'aide en ligne" grâce à l'adresse www.php.net suivi du nom de la fonction : www.php.net/echo ou www.php.net/trim. Tous les scripts de ce cours sont disponibles (en exécution et en lecture) sur le serveur linux <http://serveurlinux/CoursPHP/>.

3-2 les commentaires

PHP supporte les commentaires C : */* commentaire */*, C++ : *//commentaire* et shell Unix (et donc linux) : *#commentaire*.

3-3 l'affichage

PHP fournit 3 fonctions permettant d'envoyer du texte au navigateur. Ces fonctions ont la particularité de pouvoir insérer dans les données envoyées des valeurs variables, c'est ce qui

rend possible la création de pages dynamiques. Les 3 fonctions sont les suivantes : *void echo (string arg1 [, string argn...])*, *int print (string arg)* et *void printf (string format [, mixed args])*. Les chaînes de caractères étant délimitées par des guillemets, si on veut afficher des guillemets, un dollar ou un antislash, il faut "échapper" ces caractères en les précédant du caractère antislash (\). La fonction printf permet, comme en langage C, un formatage des données.

3-4 les variables

En PHP, les variables sont **toutes** préfixées du caractère \$ puis suivies du nom. Le nom doit commencé par une lettre (y compris les caractères ASCII 0x7f à 0xff) ou par un souligné (_) il peut ensuite comporter des lettres, des chiffres ou le caractère souligné. Le nom est sensible à la casse. Une variable est automatiquement définie lors de sa première utilisation (pas besoin de la déclarer avant de l'utiliser). De même, elle est automatiquement typée suivant la valeur avec laquelle elle est initialisée (elle peut très bien être initialisée avec une chaîne de caractères puis réinitialisée avec un entier puis avec un réel). La portée d'une variable est celle du script dans lequel elle est définie.

Exemple :

```
<?php
$variable = "valeur";
echo $variable;
echo "<br>";
echo("variable contient \"$variable\"<br>");
$variable = 123;
echo "$variable<br>";
$variable = 123.45;
print("\$variable = $variable<br>");
$pi = 3.1415926535;
$r = 2;
printf("un cercle de rayon %d a pour périmètre %f<br>", $r, 2*$pi*$r);
?>
```

Résultat :

```
valeur
variable contient "valeur"
123
$variable = 123.45
un cercle de rayon 2 a pour périmètre 12.566371
```

PHP propose une particularité : les variables dynamiques (appelées aussi variable variable). Le principe est d'utiliser une variable qui stocke le nom d'une autre variable et d'y faire référence.

Exemple :

```
<?php
$variable = 12.34;
$nom = "variable";
echo "\$variable = ", $variable, "<br>";
echo "\$nom = ", $nom, "<br>";
echo "\$\$nom = ", $$nom, "<br>";
#manière plus recommandée :
echo "\${$nom} = ", ${$nom}, "<br>";
?>
```

Résultat :

```
$variable = 12.34  
$nom = variable  
$$nom = 12.34  
${$nom} = 12.34
```

Quelques fonctions utiles pour les variables (voir les exemples sur serveurLinux) :

- une variable est elle vide : *bool empty(mixed var)*,
- une variable est elle définie : *bool isset(mixed var [,mixed var [, ...]])*,
- supprimer une variable : *void unset(mixed var [,mixed var [, ...]])*,
- obtenir des informations sur une variable : *void var_dump(mixed expression [,mixed expression [, ...]])*.

3-5 les constantes

Une constante est une variable dont la valeur est **inchangeable** lors de l'exécution d'un programme. Avec PHP, les constantes sont définies grâce à la fonction *define* suivant la syntaxe : *define("Nom_de_la_variable", Valeur)*; Le nom d'une constante ne doit pas commencer par le caractère \$ (de cette façon aucune affection n'est possible). Par convention, les constantes sont toujours en majuscules. Seuls les types de données scalaires (booléen, entier, double ou chaîne de caractères) peuvent être placés dans une constante. La fonction *bool defined(string nom)* permet de savoir si la constante *nom* est définie. La fonction *array get_defined_constants()* permet de connaître la liste de toutes les constantes définies.

3-6 les types de données

PHP propose quatre types de données scalaires :

- **nombre entier** : élément signé donc de l'ensemble Z. Il peut être spécifié en décimal, en octal (précédé de 0) ou en hexadécimal (précédé de 0x). sa taille dépend de la plateforme mais est en général de 32 bits. Lorsqu'il y a dépassement de capacité, un entier est automatiquement interprété comme un réel.
- **nombre réel** (double précision mais attention à la légère perte de précision entre autre si on veut le "caster" en int essayer *echo (int) (0.1+.07)*10*),
- **chaîne de caractères** : elle peut être spécifiée de deux manières :
 - entre apostrophes mais dans ce cas, les variables ne seront pas remplacées par leur valeur,
 - entre guillemets et dans ce cas, les variables seront remplacées par leur valeur (sous réserve d'"échapper" le caractère \$).
- **booléen** : peut prendre la valeur TRUE ou FALSE (insensible à la casse). Les valeurs suivantes sont considérées comme fausses : le booléen false, l'entier 0, le nombre réel 0.0, la chaîne vide "", la chaîne "0", un tableau vide et un objet vide (aucun élément). Toutes les autres variables sont considérées comme vraies.

et deux types de données composées :

- **tableau** : collection ordonnée de couples clé/valeur. La clé peut être un entier (comme en C) et dans ce cas le tableau est dit numérique, mais la clé peut aussi être du type chaîne et dans ce cas le tableau est dit associatif. Un même tableau peut

mélanger les deux types de clé. La valeur peut être de n'importe quel type y compris du type tableau. Dans ce cas, le tableau est multi dimensionnel.

Exemples :

```
//tableau numérique :
$tableau=array("zéro","un","deux","trois","quatre");
echo $tableau[1];
echo "<br>$tableau[1]<br>";

//création d'un tableau associatif
$tableau=array("zéro"=>0,"un"=>2,"deux"=>4,"trois"=>6);
echo $tableau["un"];
echo "<br>$tableau[un]<br>";

//création d'un tableau mélangeant les deux styles
$tableau=array(0=>"zéro",1=>"un",2=>"deux","quatre"=>4,"cinq"=>
5,"sis"=>6);
echo $tableau[1];
echo "<br>$tableau[1]<br>";
echo $tableau["sis"];
echo "<br>$tableau[sis]<br>";

//création d'un tableau multi dimensionnel
$pays=array("France","Etats Unis","Japon","Grande
Bretagne","Maroc","Russie");
$capitales=array("Paris","Washington","Tokyo","Londres","Rabat"
,"Moscou");
$monnaies=array("Euro","Dollar","Yen","Livre","Dirham","Rouble"
);
$tableau=array($pays,$capitales,$monnaies);
echo $tableau[0][0];
echo "<br>{$tableau[0][0]}<br>";
echo $tableau[1][2];
echo "<br>{$tableau[2][1]}<br>";
echo "$tableau[0]";
echo "<br>$tableau[0][0]<br>";
```

Résultat :

```
un
un
2
2
un
un
6
6
France
France
Tokyo
Dollar
Array
Array[0]
```

Dans un tableau numérique à une dimension, \$tableau[1] donne la valeur du second élément. Dans un tableau associatif à une dimension, la valeur de la clé *un* est donné par : \$tableau["un"]. Lorsque cette variable est dans une chaîne de caractères délimitée par des guillemets, on supprime les guillemets de la clé. Dans l'exemple du tableau de dimension 2, le premier indice sélectionne le tableau et le second indice l'élément dans le tableau.

Attention : il faut délimiter les variables sinon PHP comprend *\$tableau[0]* comme variable (qui retourne Array) et il affiche Array[0] !

- **objet** : lorsque l'on utilise des classes. L'objet est instancié grâce à l'opérateur new.

Note : Dans la documentation des fonctions, on rencontre souvent le type *mixed*. C'est un pseudo-type, qui indique que le paramètre peut indifféremment prendre plusieurs types.

Quelques fonctions utiles (voir les exemples sur serveurLinux) :

- convertir une variable d'un type vers un autre : *boolean set_type(mixed var, string type)* (il est aussi possible de caster),
- obtenir le type d'une variable : *string gettype(mixed var)*,
- savoir si une variable est d'un type particulier : *bool is_bool(mixed var)*, *bool is_float(mixed var)*, *bool is_real(mixed var)*, *bool is_int(mixed var)*, *bool is_integer(mixed var)*, *bool is_numeric(mixed var)*, *bool is_string(mixed var)*, *bool is_string(mixed var)*, *bool is_object(mixed var)*,
- convertir une variable en chaîne : *string strval (mixed var)*,
- convertir une chaîne en nombre réel : *float floatval (mixed var)*,
- convertir une chaîne en entier : *int intval (mixed var [, int base])*,
- éliminer les caractères "blancs" : *string trim(string str [,string charlist])*, *string rtrim(string str [,string charlist])*, *string ltrim(string str [,string charlist])*,
- connaître le nombre d'éléments d'un tableau : *int count(mixed var [,int mode])*,
- tester si une valeur est dans un tableau : *bool in_array(mixed needle, array haystack [,bool strict])*,
- rechercher une valeur dans un tableau : *mixed array_search(mixed needle, array haystack [,bool strict])*,
- trier un tableau : *bool asort (array array [,int sort_flags])*, *bool arsort(array array [, int sort_flags])*, *bool ksort(array array [, int sort_flags])*, *bool krsort(array array [, int sort_flags])*,
- découper une chaîne selon un séparateur et stocker les éléments dans un tableau : *array explode(string separator, string string [, int limit])*,
- regrouper les éléments d'un tableau dans une chaîne de caractères à l'aide d'un séparateur : *string implode(string glue, array pieces)*,
- connaître toutes les clés d'un tableau : *array array_keys(array input [,mixed search_value])*,
- ...

4 Les opérateurs

- affectation :
 - par valeur : *\$variable = 49*; variable est initialisé à 49,
 - par référence : *\$reference = \$\$variable*; reference fait référence à la variable *\$variable*. Les deux variables pointent la même zone mémoire. Une modification de l'une modifie donc l'autre.

- arithmétique :
 - somme : `$variable1 + $variable2`
 - différence : `$variable1 - $variable2`
 - produit : `$variable1 * $variable2`
 - division : `$variable1 / $variable2`
 - modulo : `$variable1 % $variable2`
 - post incrémentation : `$variable1++`
 - post décrémentement : `$variable1--`
 - pré incrémentation : `++$variable1`
 - pré décrémentement : `--$variable1`
- logique :
 - ou : `$valeur1 or $valeur2` ou `$valeur1 || $valeur2` (or est moins prioritaire que `||`)
 - et : `$valeur1 and $valeur2` ou `$valeur1 && $valeur2` (and est moins prioritaire que `&&`)
 - ou exclusif : `$valeur1 xor $valeur2`
 - négation : `!$valeur1`
 - décalage à gauche : `$valeur1 << $valeur2`
 - décalage à droite : `$valeur1 >> $valeur2`
- sur les chaînes : concaténation `$chaine1 = $chaine2 . $chaine3`
- comparaison :
 - égalité : `$valeur1 == $valeur2`
 - égalité et type identique : `$valeur1 === $valeur2`
 - différent : `$valeur1 != $valeur2`
 - inférieur : `$valeur1 < $valeur2`
 - supérieur : `$valeur1 > $valeur2`
 - inférieur ou égal : `$valeur1 <= $valeur2`
 - supérieur ou égal : `$valeur1 >= $valeur2`
- ternaire : `$valeur = (test) ? $valeur1 : $valeur2` (comme en C)
- sur les tableaux : addition `$tableau 1 + $tableau2` mais attention : si des clés sont communes aux deux tableaux, les valeurs du premier tableau sont gardées et ne sont donc pas écrasées par celles du second tableau. Les valeurs des clés de tableau2 qui n'existent pas dans tableau1 sont ajoutées à la suite de tableau1.

Comme en C, il est possible de réaliser une opération assignation du type `$valeur1 +=`
10.

5 Les structures de contrôle

En PHP, on trouve les structures (la plupart connues pour un programmeur shell, C, C++ ...) suivantes :

- if,
- if else,
- if elseif else (enchaînement de tests),
- switch,
- while,
- do while,
- for,

- **foreach** : elle permet de passer en revue un tableau et d'obtenir la valeur, et éventuellement, la clé de l'élément pointé. Dans le cas d'un tableau multi dimensionnel, il faut utiliser autant de foreach que de tableaux imbriqués.

Exemples :

```
$pays=array("France","Etats Unis","Japon","Grande
Bretagne","Maroc","Russie");
$capitales=array("Paris","Washington","Tokyo","Londres","Rabat","M
oscou");
$monnaies=array("Euro","Dollar","Yen","Livre","Dirham","Rouble");
$tableau=array($pays,$capitales,$monnaies);

echo "<table width = \"50%\" border=\"2\" cellpadding = \"3\"
cellspacing = \"5\">";
foreach($tableau as $stab)
{
    echo "<tr align=\"center\">";
    foreach($stab as $valeur)
    {
        echo "<td>$valeur</td>";
    }
    echo "</tr>";
}
echo "</table><br>";
```

Résultat :

France	Etats Unis	Japon	Grande Bretagne	Maroc	Russie
Paris	Washington	Tokyo	Londres	Rabat	Moscou
Euro	Dollar	Yen	Livre	Dirham	Rouble

- **include**, **include_once**, **require**, **require_once** : permettent d'inclure un fichier dans le script qui est exécuté. Si le fichier exécuté contient du code HTML, il est ajouté à la page en construction. Si le fichier exécuté contient du code PHP, il est exécuté. Le comportement de **include** et **require** est identique à la gestion des erreurs près (**include** génère des alertes (warning) et **require** des erreurs fatales). **include_once** et **require_once** permettent de n'inclure le fichier qu'une seule fois même s'il est appelé plusieurs fois. Il est judicieux de les utiliser lorsqu'on inclut des fichiers de définition de constantes, de fonctions ou de classes. La visibilité des variables, fonctions, classes est celle qui existerait s'il n'y avait qu'un seul fichier.

PHP permet d'incorporer du code HTML dans certaines structures de contrôle (if, while, for, foreach et switch). Dans ce cas, l'accolade ouvrante est remplacée par : et l'accolade fermante par endif, endwhile, endfor ou endswitch.

Exemples :

```
<?php
echo "<br><br><strong>code HTML à l'intérieur des structures
:</strong><br>";
$a = 49;
$b = 14;
if ($a > $b) :
//attention de na pas oublier les deux points
?>
$a = <?php echo $a ?> est plus grand que $b = <?php echo $b ?>
<br>
<?php
elseif ($a == $b) :
//attention de na pas oublier les deux points
?>
$a = <?php echo $a ?> est égal à $b = <?php echo $b ?>
<?php
else :
//attention de na pas oublier les deux points
?>
$a = <?php echo $a ?> est plus petit que $b = <?php echo $b?>
<?php
//pour indiquer la fin de la structure (endif, endfor, endwhile,
endswitch)
endif;
?>
```

Résultat :

code HTML à l'intérieur des structures

`$a = 49 est plus grand que $b = 14`

- **die** : interrompre un script. Attention : la page générée est laissée en l'état et peut donc être incohérente. Il est aussi possible d'interrompre un script avec la commande **exit**.

6 Fonctions et classes

En PHP, il est possible de définir ses propres fonctions et ses propres classes.

6-1 les fonctions

Pour définir une fonction, il faut utiliser le mot clé **function** suivi du nom de la fonction et des éventuels paramètres entre parenthèses : *function ma_fonction(\$param1, \$param2)*. Elle peut retourner une valeur grâce à l'instruction **return**. Comme il est possible de retourner un tableau, il est donc possible de retourner plusieurs valeurs, mais à la condition de les écrire dans le tableau retourné. Depuis PHP4, il est possible d'utiliser une fonction avant de l'avoir définie (dans le même script !).

- fonctions variables

Comme pour les variables variables, PHP permet l'utilisation de fonctions variables. Une variable contient le nom d'une fonction : *calculer("produit",5,8)*; la fonction calculer appelle *produit(5,8)*;

- valeur d'un paramètre par défaut

En PHP, il est possible de définir une valeur par défaut pour un paramètre :

function produit (\$var1, \$var2 = 1)

- passage de paramètre par référence

Par défaut, le passage de paramètre se fait par valeur. La modification de la valeur d'un paramètre à l'intérieur d'une fonction n'a donc aucun effet sur les valeurs du script appelant.

Il est possible de passer un paramètre par référence (on passe son adresse et non une copie de sa valeur) en le précédant de l'opérateur de référence : &. Une fonction peut aussi définir un paramètre comme étant toujours passé par référence. Le caractère & doit précéder la variable dans la définition de la fonction.

- nombre variable de paramètres

Depuis PHP4, une fonction peut admettre un nombre variable de paramètres. Les fonctions *int func_num_args(void)*, *mixed func_get_arg(int arg_num)* et *array func_get_args(void)* permettent, respectivement, de connaître le nombre d'arguments reçus, un des paramètres reçus et l'ensemble des paramètres reçus.

- visibilité des variables

Les variables qui sont définies dans une fonction sont évidemment locales à la fonction. Si on veut utiliser la variable, de même nom, du script appelant, il faut utiliser l'instruction *global* ou le tableau associatif *\$GLOBALS*.

- variable static

Si on déclare une variable *static* dans une fonction, elle ne sera pas réinitialisée à chaque appel de fonction.

6-2 les classes en PHP 5

En PHP5, il est possible de créer des classes mais pour les habitués de C++, il y a quelques différences :

- pas de ; après l'accolade fermante de la définition de la classe,
- pas d'héritage multiple,
- pas de surcharge de méthode dans une classe (possible dans les sous classes),

- constructeur

Une méthode appelée `__construct`. Pour une compatibilité avec PHP4, si cette méthode n'existe pas, une méthode du même nom que la classe est recherchée. Dans le cas où une classe dérive d'une autre classe, si la classe dérivée n'a pas de constructeur, le constructeur de la classe de base est appelé. Si la classe dérivée a un constructeur, le constructeur de la classe de base n'est pas appelé. Si on veut tout de même appeler le constructeur de la classe de base, il faut le faire explicitement : *parent::__construct()*.

- destructeur

En PHP5, il y a un destructeur : une fonction appelée `__destruct`. De même que dans le cas du constructeur, le destructeur de la classe de base n'est pas appelé. Si on veut tout de même l'appeler, il faut le faire explicitement : *parent::__destruct()*. Le destructeur est toujours appelé lorsque l'objet n'est plus référencé, c'est-à-dire suite à l'utilisation de la fonction *unset* ou à la fin du script.

- visibilité

La méthode de déclaration de variable en PHP 4 avec le mot clé *var* n'est plus valide pour les objets en PHP 5. Pour des raisons de compatibilité, une variable déclarée ainsi aura automatiquement une visibilité publique et une erreur de niveau *E_STRICT* sera générée. Les

méthodes des classes doivent être définies en tant que publiques, privées ou protégées. Les méthodes sans déclaration seront automatiquement définies comme étant publiques.

- accesseurs

Les méthodes `__get($attribut)` et `__set($attribut,$valeur)` permettent d'accéder respectivement en lecture et en écriture aux attributs de la classe.

- toString

Cette fonction, si elle est implémentée, sert à réaliser des conversions. Elle est appelée à chaque fois qu'un objet est appelé avec la fonction `print()` ou `echo()`.

- héritage

L'héritage reste simple (mais toujours pas d'héritage multiple) se fait avec le mot clé `extends`. Les interfaces faisant leur apparition, on pourra implémenter plusieurs interfaces et hériter d'une classe en même temps.

- static

Il est possible de déclarer des membres ou des méthodes comme statiques. Ceci permet de les appeler en dehors du contexte objet. La déclaration `static` doit être faite après la déclaration de visibilité. Comme les méthodes statiques sont appelables sans instance d'objet créée, la pseudo variable `$this` n'est pas disponible dans la méthode déclarée en tant que statique. On ne peut donc pas accéder à des propriétés statiques à travers l'objet en utilisant l'opérateur `->`.

- constante de classe

Il est possible de définir des valeurs constantes à l'intérieur d'une classe. Elles ne seront pas modifiables et on n'utilise pas le symbole `$` pour les déclarer ou les utiliser.

- méthodes et classes abstraites

PHP 5 introduit les classes et les méthodes abstraites. Il n'est pas autorisé de créer une instance d'une classe définie comme abstraite. Toutes les classes contenant au moins une méthode abstraite doivent également être abstraites. Pour définir une méthode abstraite, il faut simplement déclarer la signature de la méthode et ne fournir aucune implémentation. La classe qui implémente la méthode abstraite doit être définie avec la même visibilité ou une plus faible (si la méthode abstraite est définie en tant que protégée, la fonction l'implémentant doit être définie en tant que protégée ou publique).

- interfaces

Les interfaces objet permettent de créer du code qui spécifie quelles méthodes et variables une classe peut implémenter, sans avoir à définir comment ces méthodes seront gérées. Les interfaces sont définies en utilisant le mot clé `interface`, de la même façon qu'une classe standard mais sans aucun contenu de méthode. Les classes qui implémentent une interface doivent le faire en utilisant le mot clé `implements` et doivent contenir les définitions de toutes les méthodes listées dans l'interface. Les classes qui implémentent plus d'une interface doivent le faire en listant chaque interface, séparées d'une virgule. Toutes les méthodes déclarées dans une interface doivent être publiques.

- final

Avec le mot clé `final`, il est possible d'empêcher la surcharge d'une méthode dans une classe dérivée. Avec ce mot clé, il est aussi possible d'empêcher qu'une classe soit dérivée et réutilisée.

- static

Avec PHP5, ce mot clé apparaît et a le même effet que dans les autres langages objet.

- instanceof

Cet opérateur retourne vrai (ou faux) si un objet est de la classe, dérive de la classe ou implémente une interface.

- clone

Le mot clé clone a été ajouté pour réaliser la copie d'un objet. La recopie d'un objet à l'identique n'étant pas toujours souhaitable, la fonction `__clone`, si elle a été implémentée, permet de personnaliser la copie. Lorsqu'un objet est cloné, PHP5 effectue une copie superficielle de toutes les propriétés de l'objet. Toutes les propriétés qui sont des références à d'autres variables demeureront des références. Si une méthode `__clone()` est définie, la méthode `__clone()` du nouvel objet sera appelée pour permettre à chaque propriété qui doit l'être d'être modifiée. Une copie d'objet est créée en utilisant le mot clé clone. Le clonage n'utilise pas le constructeur.

- autoload

La fonction `__autoload` est appelée automatiquement lors d'une tentative d'instanciation d'une classe non déclarée. Le nom de la classe manquante est passé en argument à cette fonction qui doit donc charger la classe demandée. Ceci permet de ne charger que les classes utilisées dans le script.

- SPL - parcours d'objet

La Standard Php Library (SPL) est une extension compilée par défaut dans PHP5. C'est une collection d'interfaces et de classes qui permettent de résoudre les "problèmes" standards des itérateurs. PHP 5 fournit une façon de définir les objets de manière à ce qu'on puisse parcourir une liste de membres avec une structure `foreach`. Par défaut, toutes les propriétés visibles seront utilisées pour le parcours. Pour aller plus loin, il est possible d'implémenter l'interface interne de PHP 5 nommée *Iterator*. Ceci permet de déterminer comment l'objet doit être parcouru. Il est également possible de définir la classe de façon à ce qu'elle n'ait pas besoin de définir toutes les fonctions *Iterator* en implémentant simplement l'interface *PHP 5 IteratorAggregate*. (voir les interfaces et les classes de la SPL : <http://www.php.net/~helly/php/ext/spl/index.html>)

- typage objet

PHP 5 introduit le typage objet implicite. Les fonctions peuvent maintenant imposer aux paramètres d'être des objets en spécifiant le nom de la classe dans le prototype de la fonction. Si le paramètre ne satisfait pas les conditions imposées, une erreur fatale est émise. Le typage fonctionne aussi avec les fonctions. Le typage de paramètre ne fonctionne qu'avec les variables de type `object`. Le typage avec les types traditionnels, tels que `int` et `string`, n'est pas supporté.

- exceptions

PHP 5 a une gestion des exceptions similaire à ce qu'offrent les autres langages de programmation. Elles permettent de typer chaque erreur et de rassembler les traitements en un point (au lieu de tester constamment les retours de traitement). Une exception peut être lancée (mot clé *throw*) , essayée (mot clé *try*) et attrapée (mot clé *catch*) dans PHP. Un bloc d'essai doit inclure au moins un bloc de capture. De multiples blocs de capture peuvent être utilisés pour attraper différents types de classes. L'exception continuera après le dernier bloc de capture. Les exceptions peuvent être lancées dans les blocs de capture. Lorsqu'une exception est lancée, le code suivant le traitement ne sera pas exécuté et PHP tentera de trouver le premier bloc de capture correspondant. Si une exception n'est pas attrapée, une erreur fatale issue de PHP sera envoyée avec un message spécifiant que l'exception n'a pu être attrapée à moins qu'un gestionnaire ne soit défini avec la fonction `set_exception_handler`. Une classe *Exception* définie par l'utilisateur peut être définie en étendant la classe *Exception* interne. Si une classe étend la classe *Exception* interne et redéfinit le constructeur, il est vivement recommandé qu'elle appelle aussi `parent::__construct()` pour s'assurer que toutes les données disponibles ont été proprement assignées. La méthode `__toString()` peut être réécrite pour fournir un affichage personnalisé lorsque l'objet est présenté comme une chaîne.

7 Gestion des formulaires

7-1 rappels

Un formulaire est défini entre les balises `<form>` et `</form>` :

```
<form action = "url" method = "post" name = "nom">
</form>
```

L'option *action* indique l'url du script qui va traiter le formulaire. Si cette option n'est pas présente c'est la même url que la page utilisée. L'option *method* indique le mode de transmission des informations vers le serveur. Lorsqu'elle vaut POST, les données sont transmises dans le corps de la requête et lorsqu'elle vaut GET, les données sont transmises dans l'url (dans l'en-tête de la requête). Si cette option n'est pas présente, c'est la méthode GET qui est utilisée. L'option *name* est le nom du formulaire ce qui permet de différencier les formulaires lorsque la page en contient plusieurs (mais n'est pas intéressant dans notre cas puisqu'elle n'est pas transmise au serveur).

Entre ces deux balises, on trouve les contrôles du formulaire.

7-2 rôle de PHP dans les formulaires – récupération des données

Pour faire interagir un formulaire et un script PHP, il y a trois méthodes :

- le formulaire est une page HTML et l'option *action* de la balise *form* indique l'url du script à activer,
- le formulaire est un script PHP et l'option *action* de la balise *form* indique l'url du script à activer,
- le formulaire est un script PHP et l'option *action* de la balise *form* n'existe pas et c'est donc ce même script qui sera activé.

PHP peut donc intervenir à deux niveaux :

- lors de la construction du formulaire (construction dynamique),
- lors du traitement (toujours).

Pour pouvoir traiter les données, il faut évidemment les récupérer. Lors de la soumission du formulaire, toutes les variables sont enregistrées dans un tableau associatif nommé :

- `$_POST` si l'option *method* de la balise *form* est POST,
- `$_GET` si l'option *method* de la balise *form* est GET.

La clé de ce tableau associatif est l'option *name* de la balise du contrôle utilisé. Le tableau ne contient que les variables du formulaire qui est soumis.

Il peut exister une autre possibilité. Si PHP a été configuré avec *register_globals* à on, lors de la soumission, des variables qui héritent de l'option *name* de la balise du contrôle utilisée sont créées dans le script gérant le formulaire. Elles sont donc utilisables. Par défaut et pour des raisons de sécurité, PHP est configuré avec *register_globals* à off et il est déconseillé de modifier cette configuration. Pour connaître l'état de *register_globals*, il faut exécuter la fonction *phpinfo*.

7-3 quelques contrôles et la récupération des données

- zones contenant du texte :

C'est le cas des zones `<input>` de type *text*, *password*, *file*, *hidden* ainsi que des zones `<textarea>`. Dans ce cas, une variable associée de nom la valeur de l'option *name* est créée et contient le texte saisi.

Exemple : Pour une zone de texte déclarée comme ceci

```
<INPUT type="texte" name="nom">
```

On pourra obtenir le texte saisi par `$_POST["nom"]` ou `$_GET["nom"]`.

- groupes de boutons radio :

C'est le cas des zones `<input>` de type *radio*. Dans ce cas, une variable associée de nom la valeur de l'option *name* est créée et contient la valeur de l'option *value* du bouton radio coché.

Exemple : Pour une sélection par boutons radio déclarée comme ceci

```
<input type="radio" name="choix" value="choix1" checked>choix N°1  
<input type="radio" name="choix" value="choix2">choix N°2  
<input type="radio" name="choix" value="choix3">choix N°3  
<input type="radio" name="choix" value="choix4">choix N°4
```

On pourra connaître le bouton radio coché par `$_POST["choix"]` ou `$_GET["choix"]`. On obtiendra une des valeurs suivantes : `choix1`, `choix2`, `choix3` ou `choix4`.

- cases à cocher :

C'est le cas des zones `<input>` de type *checkbox*. Dans ce cas, il y a plusieurs méthodes pour récupérer les informations :

- mettre différentes valeurs dans l'option *name* et pas d'option *value*. Dans ce cas, si la case est cochée, une variable de nom la valeur de l'option *name* est créée et contient *on*.

```
<input type="checkbox" name="case1">case N°1  
<input type="checkbox" name="case2" checked>case N°2  
<input type="checkbox" name="case3">case N°3  
<input type="checkbox" name="case4" checked>case N°4
```

On pourra savoir si la case N°1 est cochée en testant l'existence de la variable `$_POST["case1"]` ou `$_GET["case1"]`. Si elle existe, elle est à *on*.

- mettre la même valeur (un nom de tableau) dans l'option *name* et différentes valeur dans l'option *value*. Dans ce cas, si la case est cochée une variable de nom la valeur de l'option *name* est créée et contient la valeur de l'option *value* de la (ou des) case(s) à cocher cochée(s).

```
<input type="checkbox" name="case[]" value="valeur1">case N°1  
<input type="checkbox" name="case[]" value="valeur2" checked>case  
N°2  
<input type="checkbox" name="case[]" value="valeur3">case N°3  
<input type="checkbox" name="case[]" value="valeur4" checked>case  
N°4
```


On pourra savoir si au moins une case à cocher est cochée en testant l'existence de la variable `$_POST["case"]` ou `$_GET["case"]`. Ce tableau (numérique) contiendra comme valeur l'option value des cases à cocher cochées.

- mettre différentes valeurs (un élément d'un tableau numérique ou d'un tableau associatif) dans l'option *name* et pas d'option *value*. Dans ce cas, si la case à cocher est cochée, l'élément est présent dans le tableau.

```
<input type="checkbox" name="case1[valeur1]">case N°1  
<input type="checkbox" name="case1[valeur2]" checked>case N°2  
<input type="checkbox" name="case1[valeur3]">case N°3  
<input type="checkbox" name="case1[valeur4]" checked>case N°4
```

On pourra savoir si la case N°1 est cochée en testant l'existence de la variable `$_POST["case1"]["valeur1"]` ou `$_GET["case1"]["valeur1"]`. Si au moins une case à cocher est cochée, le tableau `$_POST["case1"]` ou `$_GET["case1"]` existe.

- liste à sélection unique :

C'est le cas de la zone `<select>` sans l'option *multiple*. Dans ce cas, une variable associée de nom la valeur de l'option *name* est créée et contient la valeur de l'option *value* si elle existe et le texte affiché dans la liste sinon.

```
<select name="selection">  
<option selected>item1  
<option>item2  
<option>item3  
<option>item4  
</select>
```

On pourra connaître l'élément sélectionné grâce au contenu de la variable `$_POST["selection"]` ou `$_GET["selection"]`.

- liste à sélection multiple :

C'est le cas de la zone `<select>` avec l'option *multiple*. Dans ce cas, si au moins un item est sélectionné, une variable associée de nom la valeur de l'option *name* (un nom de tableau) est créée et contient la valeur de l'option *value* si elle existe et le texte affiché dans la liste sinon.

```
<select name="selection_multiple[]" size="4" multiple>  
<option selected>multi1  
<option>multi2  
<option selected>multi3  
<option>multi4  
</select>
```

On pourra savoir si au moins un item est sélectionné en testant l'existence de la variable `$_POST["selection_multiple"]` ou `$_GET["selection_multiple"]`. Si ce tableau numérique existe, il contient la valeur de l'option value si elle existe et le nom de l'item dans le cas contraire.

- bouton de validation :

C'est le cas de la zone `<input>` de type *submit*. Si ce bouton a une option *name*, une variable, nommée *name*, sera créée. Cette variable contiendra la valeur de l'option *value* si elle existe et le texte par défaut sinon (*Soumettre la requête*). Dans le cas où l'option *name*

n'existe pas, aucune variable n'est créée. Ceci n'est pas gênant s'il n'y a qu'un formulaire sur la page, sinon il faut nommer ces boutons.

```
<input type="submit" name="Valider">
```

On pourra savoir que ce formulaire a été soumis en testant la variable `$_POST["Valider"]` ou `$_GET["Valider"]`. Le contenu de cette variable sera *Soumettre la requête* puisque l'option `value` n'est pas présente.

- bouton image :

C'est le cas de la zone `<input>` avec le type `image`. Dans ce cas, deux variables associées de nom la valeur de l'option `name` suivi de `_x` pour une et de `_y` pour l'autre sont créées. Si le contrôle ne comporte pas l'option `name`, les variables s'appellent `x` et `y`. Elles contiennent la position relative (en pixel) du clic de la souris.

Les boutons de type `reset` ou `button` ne provoquent pas de création de variable associée. Ils n'interviennent que pour une action côté navigateur. Le premier permet de remettre la configuration d'origine.

7-4 problèmes avec les données saisies

- magic_quotes :

Si PHP est configuré avec l'option `magic_quotes_gpc` à *on*, toutes les données arrivant par une méthode GET, POST ou COOKIE sont automatiquement encodées avec un caractère `\` devant les caractères `'` et `\`. L'objectif de ceci est de favoriser l'enregistrement des données dans une base de données, mais l'inconvénient est qu'il provoque des problèmes d'affichage.

Le script suivant, qui lit le contenu d'une zone de texte et réécrit le contenu dans la même zone de texte, en est un exemple :

```
<form>
Texte : <input type="text" name="texte" value="
<?php
if (isset($_GET["texte"])) echo $_GET["texte"];
?>
">
<br>
<input type="submit">
</form>
```

Si on saisit *L'espérance* puis que l'on soumet le formulaire, on a l'affichage suivant :

Texte :

Si on soumet à nouveau le formulaire avec le texte tel qu'il est on obtient l'affichage suivant :

Texte :

Dans le premier cas, le caractère ' a été "échappé". Dans le second cas le caractère \ puis le caractère ' ont été "échappés". Cette possibilité est intéressante lorsqu'on souhaite écrire cette information dans une base de données MySQL car le caractère ' est le délimiteur de chaîne. En faisant précéder ce caractère du caractère \, on indique que l'on souhaite écrire ce caractère '. Cette possibilité est déjà moins intéressante lorsqu'on souhaite écrire dans une autre base de données : Oracle, Sybase ou SQL Server utilisent un autre caractère d'échappement : '. PHP peut s'adapter en positionnant l'option *magic_quotes_sybase*. Si elle est à off, le caractère d'échappement est \ et si elle est à on, le caractère d'échappement est '.

Mais dans tous les cas, le problème reste entier pour l'affichage. PHP propose deux fonctions :

- *string addslashes(string str)* qui retourne la chaîne, après avoir échappé tous les caractères qui doivent l'être (' " et \), pour être utilisé dans une requête de base de données.
- *string stripslashes(string str)* qui retourne une chaîne dont les anti-slash ont été supprimés.

De plus, il est possible de savoir la valeur de l'option *magic_quotes_gpc* grâce à la fonction *int get_magic_quotes_gpc(void)* qui retourne 0 si elle est désactivée et 1 si elle est activée.

Pour connaître la valeur de l'option *magic_quotes_sybase*, il faut utiliser la fonction *string ini_get(string varname)* (avec *varname* = *magic_quotes_sybase*) qui retourne la valeur de l'option de configuration en cas de succès, et false sinon.

Ceci permet donc d'écrire du code qui respectera la syntaxe imposée par la base de données à laquelle on s'adresse et qui permettra d'afficher correctement le texte venant d'un formulaire.

Dans l'exemple proposé, il faut donc remplacer la ligne

```
if (isset($_GET["texte"])) echo $_GET["texte"];
par : if (isset($_GET["texte"])) echo stripslashes($_GET["texte"]);
et encore mieux par :
if (isset($_GET["texte"]))
{
    if (get_magic_quotes_gpc())
        echo stripslashes(trim($_GET["texte"]));
    else
        echo trim($_GET["texte"]);
}
```

(la fonction *string trim(string str [, string charlist])* permet de supprimer les caractères invisibles en début et en fin de chaîne)

- interprétations du navigateur :

Lorsqu'on saisit un texte avec des guillemets exemple : *c'est un "gag"* et que l'on soumet le formulaire, voici ce qu'on obtient :

Texte :
Texte saisi : C'est un "gag"

Si on regarde le source de la page on voit ceci :

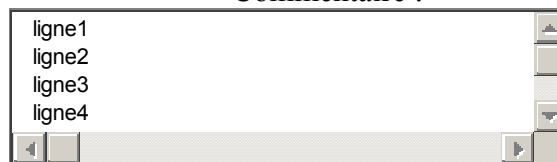
```
Texte : <input type="text" name="texte" value="
C'est un "gag">
<br>
Texte saisi : C'est un "gag"<br>
```

On peut donc voir que toute la chaîne saisie est présente mais comme le délimiteur des options est le guillemet, l'option value ne vaut que *C'est un*. Le mot gag est considéré comme une nouvelle option (que HTML ne peut pas interpréter).

Pour résoudre ce problème, on peut utiliser la fonction *string htmlspecialchars(string string [, int quote_style [, string charset]])* qui, suivant les options prises, remplace les guillemets simple ou double par leur équivalents HTML. La fonction *string htmlentities(string string [, int quote_style [, string charset]])* donnerait le même résultat. Elle est identique à la fonction *htmlspecialchars*, sauf que tous les caractères qui ont des équivalents en entités HTML sont effectivement traduits.

Si on saisit du texte sur plusieurs lignes dans une "textarea" et que l'on souhaite afficher ce texte, le navigateur ne saura pas interpréter les sauts de ligne de la zone (\n) en sauts de ligne en HTML (
).

Commentaire :



donnera l'affichage suivant :

```
Contenu de la zone de texte :
ligne1 ligne2 ligne3 ligne4
```

Pour résoudre ce problème, il faut utiliser la fonction *string nl2br(string string)* qui remplace tous les \n par des balises
.

- renvoyer sur une autre page

À la fin du traitement d'un formulaire, il peut être intéressant d'afficher une autre page. Pour cela, il est possible d'utiliser la fonction *void header (string string [, bool replace [, int http_response_code]])* avec l'option location qui permet de préciser une URL :

```
header("location: accueil.php");
```

Attention : cette fonction doit être appelée avant toute construction de page c'est-à-dire avant toute balise HTML ou avant tout affichage émanant d'un script PHP.

8 Identification - Passage de données entre pages

Lorsqu'on utilise PHP, on souhaite évidemment que le site soit interactif. Dans ce cas, on a souvent besoin d'identifier un utilisateur et de conserver des informations relatives à cet utilisateur. Une variable ayant la durée de vie d'un script, il va falloir passer ces données entre les différentes pages. La durée de la navigation continue d'un utilisateur sur un site est appelée session.

8-1 identification d'un utilisateur

Il y a deux possibilités pour identifier un utilisateur : utiliser un formulaire ou utiliser les fonctions d'authentification HTTP. Attention : les fonctions d'identification HTTP de PHP ne sont disponibles que si PHP est exécuté comme module Apache, et non pas sous la forme d'un CGI. Sous cette forme, il est possible d'utiliser la fonction `header()` pour demander une identification au client, générant ainsi l'apparition d'une fenêtre de demande d'utilisateur et de mot de passe. Une fois que les champs ont été remplis, l'URL sera de nouveau appelée, avec les variables `$_SERVER["PHP_AUTH_USER"]`, `$_SERVER["PHP_AUTH_PW"]` et `$_SERVER["PHP_AUTH_TYPE"]` contenant respectivement le nom d'utilisateur, le mot de passe et le type d'identification (actuellement, seule l'identification simple ("Basic") est supportée).

8-2 identifiant unique

Lorsque l'on gère des sessions, il peut être intéressant de créer un identifiant unique afin de matérialiser cette session. La fonction `string uniqid(string prefix [, bool lcg])` retourne un identifiant préfixé (par `prefix`) unique, basé sur l'heure courante (en micro-secondes). Le préfixe peut servir à identifier facilement différents hôtes et peut être généré de manière aléatoire.

Il est aussi possible de crypter cet identifiant unique afin que deux identifiants successifs soient plus différents. La fonction `string md5(string str [, bool raw_output])` calcule le "MD5" de la chaîne de caractères `str` en utilisant l'algorithme RSA (Data Security, Inc. MD5 Message-Digest Algorithm), et retourne un nombre de 32 caractères hexadécimaux.

8-3 passage de données entre pages

- par l'URL

Comme lorsqu'on utilise la méthode GET pour un formulaire, on va pouvoir passer des informations par l'URL. Exemple en utilisant un lien : `<a ref="page2.php?nom=< ?php echo $Nom; ?>">Page2</a>` (il est aussi possible d'utiliser la même technique avec un formulaire).

Attention :

- le problème "magic_quotes" peut à nouveau survenir (voir 7-4 problèmes avec les données saisies),
- si la valeur à transmettre comporte les caractères `?` ou `&`, ces derniers seront interprétés car considérés comme séparateur. Pour éviter ceci, il faut utiliser la fonction `string rawurlencode(string str)` qui retourne une chaîne dont tous les caractères non-alpha-numériques auront été remplacés par des séquences `%xy`. Ce codage évite que les caractères spéciaux soient interprétés comme des délimiteurs, ceci afin de protéger les URL lors du transfert.

Dans chaque script concerné par la gestion des sessions, il faudra commencer par vérifier que le script a été appelé avec un identifiant et si ce n'est pas le cas, rediriger l'utilisateur vers la page d'identification.

Problème : un utilisateur qui appelle l'URL de la manière suivante :

`http://.../page1.php?session=a1b2c3`

sera considéré comme identifié alors qu'il n'est pas passé par la page d'identification.

- par une zone de formulaire cachée

Pour chaque variable que l'on souhaite passer, on va ajouter un contrôle caché de ce type : `<input type="hidden" name="nom" value="<?php echo $Nom ?>">`.

Avec cette méthode, on n'a plus le problème des caractères ? et & mais le problème "magic_quotes" peut toujours survenir.

Problème : un utilisateur qui appelle l'URL de la manière suivante :

`http://.../page1.php?session=a1b2c3`

sera toujours considéré comme identifié alors qu'il n'est pas passé par la page d'identification mais il ne pourra pas transmettre les informations qui transitent par le formulaire caché.

- par les cookies

Un cookie est un petit fichier qui contient des informations et qui est déposé sur le poste du client. Les cookies sont automatiquement renvoyés au serveur WEB, par le navigateur, lorsque l'utilisateur navigue dans les pages du site.

La fonction `bool setcookie(string name [, string value [, int expire [, string path [, string domain [, int secure]]]])` définit un cookie qui sera envoyé avec le reste des en-têtes. Les cookies doivent passer avant tout autre en-tête et donc cela impose d'appeler cette fonction avant toute balise HTML ou tout code PHP générant un affichage. Tous les arguments sauf *name* (nom du cookie) sont optionnels. Si seul le nom est présent, le cookie portant ce nom sera supprimé du navigateur de l'internaute. Le paramètre *expire* détermine la date d'expiration de ce cookie. Si ce paramètre n'est pas spécifié (ou vaut 0), le cookie expire à la fin de la session et le fichier n'est pas visible sur le poste du client.

Attention : si l'utilisateur refuse les cookies, la fonction `setcookie` retourne true mais le cookie n'est pas déposé. Pour savoir si l'utilisateur accepte les cookies, il faut en déposer un puis lorsque l'utilisateur rechargera une page tester son existence.

Lors de la réception du cookie, toutes les variables sont enregistrées dans un tableau associatif nommé : `$_COOKIE`. La clé de ce tableau associatif est le nom du cookie. Ce tableau contient tous les cookies du site (pas seulement ceux de la page actuelle).

Avec cette méthode, on n'a évidemment pas le problème des caractères ? et & mais le problème "magic_quotes" peut toujours survenir.

Les cookies pourraient être pratiques pour gérer les sessions mais comme ils ne sont pas nécessairement acceptés par tous les clients, cette méthode n'est pas intéressante pour un site public.

9 Gestion des sessions

9-1 présentation

Les techniques précédentes permettent de transmettre des informations d'une page à une autre mais la sécurité n'est pas toujours garantie et nécessite parfois beaucoup de codage. PHP4 a introduit la gestion de session qui permet de mémoriser des informations pendant toute la session et de sécuriser les connexions. Une session est constituée d'un identificateur unique attribué à chaque connexion sur le site. PHP s'arrange pour le transmettre d'une page à une autre en utilisant les cookies ou l'URL (une des techniques vues précédemment). On peut sauvegarder les variables que l'on souhaite. Elles sont sauvegardées du côté du serveur et sont disponibles pendant toute la durée de la session grâce au tableau associatif `$_SESSION`. Le problème "magic_quotes" n'existe plus avec les variables de session.

9-2 fonctions PHP gérant les sessions

La fonction *bool session_start(void)* crée, ou restaure une session trouvée, sur le serveur. Cette fonction doit être appelée avant toute balise HTML ou tout code PHP générant un affichage.

Pour enregistrer une variable, il faut utiliser le tableau associatif `$_SESSION`. `$_SESSION["ma_variable"] = sa_valeur;` mémorise une variable nommée `ma_variable` qui est initialisée à `sa_valeur`. Les variables sont enregistrées à la fin du script quel que soit l'endroit (à l'intérieur du script) où elles sont modifiées.

Pour savoir si une variable fait partie de la session courante, il faut utiliser la fonction *bool isset(mixed \$var [, mixed \$var [, \$...]])* sur la variable du tableau associatif `$_SESSION`. Cette fonction doit être appelée après l'appel de la fonction *session_start*.

La fonction *void unset(mixed \$var [, mixed \$var [, mixed \$...]])* sur une variable du tableau associatif `$_SESSION` permet de supprimer cette variable de la session courante mais elle reste disponible pour le script en cours puisque la mémorisation a lieu à la fin du script. Cette fonction doit être appelée après l'appel de la fonction *session_start*.

La fonction *void session_unset(void)* détruit toutes les variables de la session courante. Cette fonction doit être appelée après l'appel de la fonction *session_start*.

La fonction *bool session_destroy(void)* détruit la session courante et toutes les données associées.

Attention : Cette fonction ne détruit pas les variables globales associées avec la session, de même, elle ne détruit pas le cookie de session (s'il existe).

La fonction *string session_id([string id])* retourne l'identifiant courant de session.

Attention : pour tester si un utilisateur s'est correctement identifié, il ne faut pas utiliser cette fonction car cet utilisateur peut refuser les cookies et demander une page en passant n'importe quel identifiant. Dans ce cas, la fonction *session_id* retournera cet identifiant et vous pourriez considérer qu'il s'est authentifié. Pour être sûr, il faut tester l'existence d'une variable de session grâce à la fonction *isset*.

La fonction `string session_name([string name])` retourne le nom de la session courante (PHPSESSID par défaut). Si `name` est fourni, le nom de la session courante sera remplacé par cette valeur.

Attention : Le nom de session est toujours remis à sa valeur par défaut. Il faut donc utiliser `session_name` au début de chaque script, et notamment avant `session_start`.

9-3 gestion par URL ou par cookies

Deux paramètres de configuration gèrent la transmission de l'identificateur :

- `session.use_cookie` : qui (lorsqu'il est positionné à 1) tente d'utiliser les cookies pour transmettre l'identificateur ou qui n'utilise pas les cookies (lorsqu'il est à 0),
- `session.use_trans_sid` : qui (lorsqu'il est à 0) n'utilise pas l'URL pour transmettre l'identificateur ou qui utilise l'URL (lorsqu'il est à 1) si l'identificateur ne peut pas être transmis par cookie (`session.use_cookie` à 0 ou refus des cookies par l'utilisateur).

Les différents cas possibles provoquent différents fonctionnements. Par défaut, Apache est configuré avec `session.use_cookie` positionné et `session.use_trans_sid` non positionné.

Nous allons les étudier à travers la page suivante :

```
<?php
session_start();
$_SESSION['Nom'] = "MARTIN";
$_SESSION['Prenom'] = "Arthur";
echo "<br>Nom de la session : " . session_name() . "<br>";
echo "Identificateur de la session : " . session_id() . "<br><br>";
$_SESSION['Fonction'] = "Directeur d'&eacute;ttablissement";
$_SESSION['Societe'] = "MM Corporation & Co";
echo "Pr&eacute;n&eacute;m : " . $_SESSION['Prenom'] . "<br>";
echo "Nom : " . $_SESSION['Nom'] . "<br>";
echo "Fonction : " . $_SESSION['Fonction'] . "<br>";
echo "Soci&eacute;t&eacute; : " . $_SESSION['Societe'] . "<br>";
//pour voir la constante SID :
//echo "SID : " . SID . "<br>";
?>
<body bgcolor="ffff00">
<br><br>
<a href="session2.php">Voir les variables de session</a>
</body>
```

Qui provoque l'affichage suivant :

Nom de la session : PHPSESSID
Identificateur de la session : 80acc4e7a28c4c60ae8b9f173a3fc6c9

Prénom : Arthur
Nom : MARTIN
Fonction : Directeur d'établissement
Société : MM Corporation & Co

[Voir les variables de session](#)

Si l'utilisateur accepte les cookies, lors de l'appel de la page permettant de voir les variables de session (clic sur le lien et appel de la page session2.php) le cookie sera envoyé au serveur. Lors de l'exécution de la fonction `session_start` de la page session2.php le serveur va récupérer le même identificateur de session. Les variables de session seront donc connues et disponibles jusqu'à la destruction de la session. La session fonctionne normalement.

Si l'utilisateur n'accepte pas les cookies, lors de l'appel de la page permettant de voir les variables de session (clic sur le lien), le serveur ne recevra pas de cookie et la fonction `session_start` de la page session2.php va générer un nouvel identificateur pour lequel les variables de session n'existeront pas. La session ne fonctionne pas.

Il existe une constante appelée `SID` que l'on peut utiliser. Cette constante est fabriquée à partir des fonctions `session_name` et `session_id` et elle permet de transmettre l'identificateur de session par l'URL. Cette constante `SID` n'est pas renseignée si le cookie est accepté sauf lors de la première page où PHP ne sait pas encore si l'utilisateur accepte ou non les cookies.

Il devient donc possible de gérer des sessions lorsque l'utilisateur refuse les cookies. Il faut pour cela compléter les liens par `SID` si cela est nécessaire. Le code de la page 24 devient le suivant :

```
<?php
session_start();
$_SESSION['Nom'] = "MARTIN";
$_SESSION['Prenom'] = "Arthur";
echo "<br>Nom de la session : " . session_name() . "<br>";
echo "Identificateur de la session : " . session_id() . "<br><br>";
$_SESSION['Fonction'] = "Directeur d'&eacute;tablissement";
$_SESSION['Societe'] = "MM Corporation & Co";
echo "Pr&eacutenom : " . $_SESSION['Prenom'] . "<br>";
echo "Nom : " . $_SESSION['Nom'] . "<br>";
echo "Fonction : " . $_SESSION['Fonction'] . "<br>";
echo "Soci&eacute;t&eacute; : " . $_SESSION['Societe'] . "<br>";
?>
<body bgcolor="ffffc0">
<br><br>
<a href="session2_sid.php
<?php
if ((get_cfg_var("session.use_trans_sid") == 0) && (SID != ""))echo "?" . SID;
?>
">Voir les variables de session</a>
</body>
```

Le code HTML généré par cette page est le suivant :

```
<br>
Nom de la session : PHPSESSID<br>
Identificateur de la session : 387436b911d918d8d5ae56bc16191949<br><br>
Pr&eacutenom : Arthur<br>
Nom : MARTIN<br>
Fonction : Directeur d'&eacute;tablissement<br>
Soci&eacute;t&eacute; : MM Corporation & Co<br>
<body bgcolor="ffffc0"><br><br>
<a href="session2_sid.php?PHPSESSID=387436b911d918d8d5ae56bc16191949">Voir les
variables de session</a>
</body>
```

On peut donc voir que l'identificateur est passé par l'URL. Ceci est vrai que l'utilisateur accepte ou non les cookies. Le serveur ne peut pas encore savoir si l'utilisateur les accepte ou non et prévoit le cas où ils ne le seraient pas. Si on rafraichit cette page, le serveur saura si l'utilisateur accepte ou non ces cookies. S'il les accepte, il ne complètera plus l'URL. Par contre s'il ne les accepte pas, il continuera toujours à compléter l'URL afin de garder l'identificateur de la session et ainsi garder les variables de session définies. Si au lieu de rafraichir la page, on clique sur le lien, le fonctionnement sera le même : si le serveur reçoit le cookie, il ne complète plus l'URL, si le serveur ne le reçoit pas, il continue de compléter l'URL. Maintenant, la session fonctionne quel que soit le mode de fonctionnement du navigateur de l'utilisateur et ce jusqu'à ce que la session soit détruite.

Lorsque le paramètre de configuration *session.use_trans_id* est positionné à 1, le complément de l'URL est fait automatiquement (sans utiliser la constante SID).

Les autres configurations du serveur WEB donnent les fonctionnements suivants :

session.use_cookie	session.use_trans_sid	cookie client	Fonctionnement
0	0	Sans importance (le serveur n'utilise pas les cookies)	Gestion possible des sessions en utilisant la constante SID l'identificateur ne peut être transmis que par l'URL.
0	1	Sans importance (le serveur n'utilise pas les cookies)	Gestion possible des sessions : l'identificateur est transmis automatiquement par l'URL. Il n'est pas nécessaire d'utiliser la constante SID.
1	1	Accepté	Gestion de session par le cookie puisqu'il est accepté.
1	1	refusé	Gestion de session par l'URL puisque le cookie est refusé. Il n'est pas nécessaire d'utiliser la constante SID.

9-4 autres directives

Il existe d'autres paramètres de configuration pour la gestion des sessions :

- *session.save_path* : qui indique le chemin où seront sauvegardées les sessions,
- *session.cookie_lifetime* : qui impose la durée de vie d'un cookie (0 par défaut c'est à dire jusqu'à ce que le navigateur soit éteint ce qui est bien adapté à une gestion de session),
- *session.cache_limiter* : qui détermine le comportement à utiliser avec les pages de session. Les valeurs possibles sont : none, nocache (valeur par défaut), private, private_no_expire, publicpublic,
- *session_cache_expire* : qui spécifie la durée de vie des données de sessions, en minute. Cette option n'a aucune conséquence sur le contrôle de cache. Par défaut, il vaut 180 (3 heures),
- *url_rewriter.tags* : qui spécifie quelles sont les balises HTML qui doivent être réécrites si le support transparent du SID est activé. Par défaut, il vaut a=href,area=href,frame=src,input=src,form=,fieldset=.

10 Accès aux bases de données

PHP permet un interfaçage avec un grand nombre de bases de données (MySQL, Oracle 8, PostgreSQL, MS SQL Server, Sybase, DBM, Informix). Lorsqu'une base de données n'est pas directement supportée par PHP, il est possible d'utiliser un driver ODBC, pilote standard, pour communiquer avec les bases de données. La communication avec les bases de données se fait à l'aide de requêtes SQL langage reconnu par l'ensemble des SGBD.

La suite de ce document va se limiter à quelques fonctions permettant de travailler avec trois bases de données : MySQL, PostgreSQL et MS SQL Server.

10-1 connexion à une base de données

Avant toute chose, il faut se connecter à une base de données. Pour les trois bases citées précédemment, les fonctions sont les suivantes :

- *resource mysql_connect([string server [, string username [, string password [, bool new_link [, int client_flags]]]])* qui établit une connexion à un serveur MySQL,
- *resource pg_connect(string connection_string)* qui ouvre une connexion à un serveur PostgreSQL,
- *int mssql_connect([string servername [, string username [, string password]])* qui établit une connexion à un serveur MS SQL.

La connexion se ferme lorsqu'on le demande ou à la fin du script, ce qui fait qu'on n'est pas obligé de fermer explicitement la connexion à la fin du script.

Il existe des connexions dites persistantes qui ne se ferment pas à la fin du script. Elles ne donnent aucune fonctionnalité de plus que les connexions non persistantes mais permettent d'accélérer les accès à une base de données si le traitement de connexion à cette base est long. Ce temps dépend de nombreux facteurs : le type de base de données, est-elle ou non sur le même serveur, le serveur de base de données est-il chargé, etc... À l'inverse, si le nombre maximal de connexions risque d'être atteint par les connexions persistantes, la connexion suivante ne sera pas honorée.

Pour ouvrir une connexion persistante, il faut utiliser une des fonctions suivantes (en fonction de la base) :

- *resource mysql_pconnect([string server [, string username [, string password [, int client_flags]]]])* pour une base de données MySQL,
- *resource pg_pconnect(string connection_string)* pour une base PostgreSQL,
- *int mssql_pconnect([string servername [, string username [, string password]])* pour une base MS SQL.

Pour que la connexion persistante puisse s'ouvrir, il faut que le paramètre de configuration *mysql.allow_persistent* (ou *mssql.allow_persistent* ou *pgsql.allow_persistent*) soit à on. Le nombre de connexions persistantes sera limité par le paramètre *mysql.max_persistent* (ou *mssql.max_persistent* ou *pgsql.max_persistent*) et le nombre total de connexions par le paramètre *mysql.max_links* (ou *mssql.max_links* ou *pgsql.max_links*).

10-2 déconnexion d'une base de données

Si on souhaite se déconnecter d'une base de données, il faut utiliser une des fonctions suivantes :

- *bool mysql_close([resource link_identifieur])* pour MySQL,
- *bool pg_close(resource connection)* pour PostgreSQL,
- *bool mssql_close([resource link_identifieur])* pour MS SQL.

Rappel : il n'est pas nécessaire de fermer les connexions non persistantes aux bases de données, car elles seront fermées automatiquement à la fin du script.

Ces fonctions ne ferment pas les connexions persistantes !

10-3 choix de la base de données

Afin de choisir la base de données sur laquelle on veut travailler, il faut utiliser une des fonctions suivantes :

- *bool mysql_select_db(string database_name [, resource link_identifieur])* pour MySQL,
- *bool mssql_select_db(string database_name [, resource link_identifieur])* pour MS SQL.

Toutes les requêtes suivantes seront effectuées sur la base de données sélectionnée. Dans le cas de PostgreSQL, la base est sélectionnée dans la chaîne de connexion avec la fonction *pg_connect*.

10-4 exécution d'une requête

Pour exécuter une requête sur la base sélectionnée, il faut utiliser une des fonctions suivantes :

- *resource mysql_query(string query [, resource link_identifieur])* qui envoie une requête SQL à la base de données actuellement active sur le serveur MySQL. Seules les requêtes SQL SELECT, SHOW, EXPLAIN et DESCRIBE retournent une ressource de résultat MySQL, ou FALSE en cas d'échec. Pour les autres types de requêtes SQL, *mysql_query()* retourne TRUE en cas de succès et FALSE en cas d'erreur.
- *resource pg_query(resource connection, string query)* qui envoie une requête SQL à la base de données actuellement active sur le serveur PostgreSQL. Elle retourne une ressource de résultat si la requête a pu être exécutée et FALSE en cas d'échec.
- *resource mssql_query(string query [, resource link_identifieur [, int batch_size]])* qui envoie une requête SQL à la base de données actuellement active sur le serveur MS SQL. Elle retourne un identifiant positif de résultat en cas de succès ou FALSE en cas d'échec.

Attention : La requête ne doit pas être terminée par un point-virgule.

Lorsqu'une de ces fonctions est exécutée, (et entre autre avec une requête SQL SELECT), aucune donnée n'est retournée. Pour obtenir les données, il faut extraire les lignes du résultat retourné.

Dans le cas du serveur PostgreSQL, les fonctions suivantes sont intéressantes :

- *bool pg_insert(resource connection, string table_name, array assoc_array [, int options])* qui insère le tableau *assoc_array* (bâti sur le modèle *champ=>valeur* dans la table *table_name*.
- *mixed pg_delete(resource connection, string table_name, array assoc_array [, int options])* qui efface les lignes de la table *table_name*, spécifiées par le tableau associatif *assoc_array*, qui est bâti sous la forme *champ=>valeur*.
- *bool pg_insert(resource connection, string table_name, array assoc_array [, int options])* qui insère le tableau *assoc_array* (bâti sur le modèle *champ=>valeur* dans la table *table_name*.
- *array pg_select(resource connection, string table_name, array assoc_array [, int options])* qui effectue une requête de SELECT avec les lignes spécifiées par le tableau *assoc_array*, au format *champ=>valeur*. Lorsque la requête réussit, *pg_select()* retourne un tableau contenant toutes les lignes et champs qui vérifient la condition spécifiée par *assoc_array*.

10-5 extraction des données – notion de "fetch"

Lors de l'exécution d'une requête SELECT, le résultat est mis dans un buffer en mémoire et un pointeur interne est positionné sur la première ligne. L'instruction "fetch" permet de lire la ligne courante et positionne le pointeur sur la ligne suivante. Cette instruction est utilisée par certaines fonctions qui permettent d'obtenir des informations sur un résultat. Elles sont décrites (ci-après) par serveur de base de données.

- Serveur MySQL

La fonction *int mysql_num_rows(resource result)* retourne le nombre de lignes d'un résultat. Cette commande n'est valide que pour les requêtes SELECT. Pour connaître le nombre de lignes retournées par les requêtes INSERT, UPDATE ou DELETE, il faut utiliser la fonction *int mysql_affected_rows([resource link_identifier])*.

Les fonctions *array mysql_fetch_array(resource result [, int result_type])*, *array mysql_fetch_assoc(resource result)* et *array mysql_fetch_row(resource result)* retournent la ligne courante sous la forme d'un tableau ou FALSE s'il ne reste rien à lire.

La fonction *mysql_fetch_assoc* retourne un tableau associatif dont la clé est le nom de la colonne.

La fonction *mysql_fetch_row* retourne un tableau dont l'indice est le numéro de colonne (0 pour la première colonne).

La fonction *mysql_fetch_array* retourne un tableau dont le type dépend du second paramètre :

- **MYSQL_NUM** : tableau d'indices (comme la fonction *mysql_fetch_row*),
- **MYSQL_ASSOC** : tableau associatif (comme la fonction *mysql_fetch_assoc*),
- **MYSQL_BOTH** : qui donne les deux à la fois (valeur par défaut).

Enfin la fonction *object mysql_fetch_object(resource result)* retourne un objet dont les propriétés correspondent à une ligne du résultat ou FALSE s'il ne reste plus rien à lire. L'argument *result* peut prendre les valeurs **MYSQL_ASSOC**, **MYSQL_NUM** et **MYSQL_BOTH** mais dans les deux derniers cas, les données pointées par les indices ne seront pas disponibles puisqu'une variable ne peut pas commencer par un chiffre (dans le cas des objets, les données seront lues par *\$object->cle*) !

Toutes ces fonctions sont équivalentes en terme de temps d'exécution mais les tableaux associatifs et les objets apportent un confort d'utilisation puisque les clés ou nom de champs correspondent aux noms des colonnes.

- Serveur PostgreSQL

La fonction *int pg_num_rows(resource result)* retourne le nombre de lignes d'un résultat. Cette commande n'est valide que pour les requêtes SELECT. Pour connaître le nombre de lignes retournées par les requêtes INSERT, UPDATE ou DELETE, il faut utiliser la fonction *int pg_affected_rows([resource result])*.

Les fonctions *array pg_fetch_array(resource result [, int row [, int result_type]])*, *array pg_fetch_assoc(resource result [, int row])* et *array pg_fetch_row(resource result [, int row])* retournent la ligne courante sous la forme d'un tableau ou FALSE s'il ne reste rien à lire.

La fonction *pg_fetch_assoc* retourne un tableau associatif dont la clé est le nom de la colonne.

La fonction *pg_fetch_row* retourne un tableau dont l'indice est le numéro de colonne (0 pour la première colonne).

La fonction *pg_fetch_array* retourne un tableau dont le type dépend du second paramètre :

- PGSQL_NUM : tableau d'indice (comme la fonction *mysql_fetch_row*),
- PGSQL_ASSOC : tableau associatif (comme la fonction *mysql_fetch_assoc*),
- PGSQL_BOTH : qui donne les deux à la fois (valeur par défaut).

La fonction *object pg_fetch_object (resource result [, int row [, int result_type]])* retourne un objet dont les propriétés correspondent à une ligne du résultat ou FALSE s'il ne reste plus rien à lire. L'argument *result* peut prendre les valeurs PGSQL_ASSOC, PGSQL_NUM et PGSQL_BOTH mais dans les deux derniers cas, les données pointées par les indices ne seront pas disponibles puisqu'une variable ne peut pas commencer par un chiffre (dans le cas des objets, les données seront lues par \$object->cle) !

Enfin, la fonction *array pg_fetch_all (resource result)* retourne un tableau qui contient toutes les lignes et retourne FALSE s'il n'y a plus de lignes disponibles.

Toutes ces fonctions sont équivalentes en terme de temps d'exécution mais les tableaux associatifs et les objets apportent un confort d'utilisation puisque les clés ou nom de champs correspondent aux noms des colonnes.

- Serveur MS SQL

La fonction *int mssql_num_rows(resource result)* retourne le nombre de lignes d'un résultat. Cette commande n'est valide que pour les requêtes SELECT. Il n'existe pas de fonction *mssql_affected_rows*, mais pour connaître le nombre de lignes retournées par les requêtes INSERT, UPDATE ou DELETE, on peut utiliser la requête "SELECT @@ROWCOUNT" qui permet de connaître le nombre de lignes affectées par le dernier ordre de mise à jour.

Les fonctions *array mssql_fetch_array(resource result [, int result_type])* et *array mssql_fetch_row(resource result)* retournent la ligne courante sous la forme d'un tableau ou FALSE s'il ne reste rien à lire.

La fonction *mssql_fetch_row* retourne un tableau dont l'indice est le numéro de colonne (0 pour la première colonne).

La fonction *mssql_fetch_array* retourne un tableau où chaque ligne est présente deux fois. Une fois avec un indice numérique et une fois avec une clé (tableau associatif). La clé est le nom de la colonne.

Enfin la fonction *object mssql_fetch_object(resource result)* retourne un objet dont les propriétés correspondent à une ligne du résultat ou FALSE s'il ne reste plus rien à lire. Comme avec la fonction *mssql_fetch_array*, chaque ligne est présente deux fois. Les données pointées par les indices numériques ne seront pas disponibles puisqu'une variable ne peut pas commencer par un chiffre (dans le cas des objets, les données seront lues par *\$object->cle*) !

Toutes ces fonctions sont équivalentes en terme de temps d'exécution mais les tableaux associatifs et les objets apportent un confort d'utilisation puisque les clés ou nom de champs correspondent aux noms des colonnes.

10-6 gestion des erreurs

Pour tous les serveurs, il est possible de gérer les erreurs grâce à un message d'erreur. Pour obtenir ce message d'erreur, il faut utiliser une des fonctions suivantes :

- *string mysql_error([resource link_identifier])* pour le serveur MySQL. Il dispose aussi de la fonction *int mysql_errno([resource link_identifier])* qui retourne le numéro de la dernière erreur.
- *string pg_last_error([resource connection])* pour le serveur PostgreSQL (ou *string pg_result_error(resource result)* qui retourne le message d'erreur associé au résultat result. Par conséquent, l'utilisateur a des chances d'obtenir un message d'erreur plus approprié que via *pg_last_error*).
- *string mssql_get_last_message(void)* pour le serveur MS SQL. Il n'existe pas de fonction *mssql_errno*, mais pour connaître le numéro de l'erreur, on peut utiliser la requête "SELECT @@ERROR" (exécuter la requête puis extraire l'information).

10-7 magic_quotes

Lorsqu'on écrit dans une base de donnée, il faut que les données de type texte soient "échappées". Pour cela il existe la fonction *string addslashes(string str)* qui permet d'échapper tous les caractères qui doivent l'être afin que la chaîne puisse être utilisée dans une requête de base de données mais cette fonction dépend de la configuration de PHP (options *magic_quotes_gpc*, *magic_quotes_runtime* et *magic_quotes_sybase*).

Les fonctions suivantes sont intéressantes car elles permettent d'être indépendantes de la configuration :

- *string mysql_escape_string(string unescaped_string)* pour un accès à une base MySQL,
- *string pg_escape_string(string data)* pour un accès à une base PostgreSQL.

11 Gérer des fichiers

11-1 fichiers locaux

PHP permet de gérer les fichiers et les répertoires qui se trouvent sur le serveur. Les fonctions suivantes sont parmi les plus usuelles :

- *resource fopen(string filename, string mode [,bool use_include_path [,resource zcontext]])* qui crée une ressource nommée, spécifiée par le paramètre filename, sous la forme d'un flux. Si filename est de la forme "protocole://...", filename est supposé être une URL, et PHP va rechercher un gestionnaire de protocole adapté pour lire ce fichier. Si PHP décide que le fichier filename est un fichier local, il va essayer d'ouvrir un flux avec ce fichier. Le fichier doit être accessible à PHP (notions de droits d'accès à ce fichier),
- *bool fclose(resource handle)* qui ferme le fichier handle,
- *string fread(resource handle, int length)* qui lit jusqu'à length octets dans le fichier référencé par handle.
- *int fwrite (resource handle, string string [, int length])* qui écrit le contenu de la chaîne string dans le fichier pointé par handle. Si la longueur length est fournie, l'écriture s'arrêtera après length octets, ou à la fin de la chaîne.

Il existe beaucoup d'autres fonctions : basename, chgrp, chmod, chown, copy, delete, dirname, feof, fflush, fgetc, fgets, file_exists, file, filesize, filetype, fputs, fseek, fstat, is_dir, is_executable, is_file, is_link, is_readable, is_writable, mkdir, pathinfo, rename, rmdir, stat, umask, unlink ...

Remarque : si le serveur est sous Linux, l'accès aux fichiers permet de gérer le matériel via un pilote (liaison série par exemple) sous réserve d'avoir les droits.

11-2 upload

PHP est capable de recevoir des fichiers, textes ou binaires, émis par un navigateur. Un écran de chargement de fichiers peut être constitué en créant un formulaire de la manière suivante :

```
<FORM METHOD="POST" ENCTYPE="multipart/form-data">
<br><br>
<INPUT TYPE="hidden" NAME="MAX_FILE_SIZE" VALUE="1024">
Fichier : <INPUT TYPE="file" NAME="fichier">
<BR><br>
<INPUT TYPE="submit" NAME="OK" VALUE="OK">
</FORM>
```

L'option MAX_FILE_SIZE cachée doit précéder le nom du fichier à charger, et représente la taille maximale du fichier à charger. La valeur est donnée en octets. Elle n'est là qu'au titre de conseil au navigateur et (d'après la documentation) il ne faut compter sur le navigateur pour respecter cette configuration !

Le tableau associatif `$_FILES` contient toutes les données sur le fichier téléchargé. En supposant que le nom de la variable du fichier téléchargé est `userfile`, les données seront les suivantes :

- `$_FILES["userfile"]["name"]` : le nom original du fichier (sur la machine du client web),
- `$_FILES["userfile"]["type"]` : le type MIME du fichier (si le navigateur a fourni cette information),
- `$_FILES["userfile"]["size"]` : la taille, en octets, du fichier téléchargé,
- `$_FILES["userfile"]["tmp_name"]` : le nom temporaire du fichier qui sera chargé sur la machine serveur,
- `$_FILES["userfile"]["error"]` : le code d'erreur associé au téléchargement de fichier.

Le fichier téléchargé sera stocké temporairement dans le dossier temporaire du système, à moins qu'un autre dossier soit fourni avec la directive `upload_tmp_dir` dans le fichier `php.ini`.

Important : Le fichier temporaire sera automatiquement à la fin du script s'il n'a pas été déplacé ou renommé.

11-3 download

Il est possible d'utiliser un lien pour télécharger un fichier à partir du serveur mais vous ne pourrez pas envoyer des fichiers interprétés par le serveur (script PHP) ou par le navigateur (page HTML).

On peut forcer le navigateur à proposer une boîte de dialogue d'enregistrement et ceci quel que soit le type de fichier à télécharger. Pour cela on utilise certains en-têtes particuliers et au minimum :

- *Content-Disposition: attachment; filename=* qui suggère au navigateur de traiter le document comme une pièce jointe et de proposer à l'utilisateur de l'enregistrer sous le nom défini par *filename*,
- *Content-Type* : qui indique le type MIME du fichier.

puis on envoie le fichier (`readfile` ou association `fopen`, `fread`, `fclose` puis `echo`).

Exemple (plus complet) le nom du fichier est contenu dans la variable `$fichier` :

```
$info = pathinfo($fichier);
switch ($info["extension"])
{
    case "php" : $type="application/php";break;
    case "inc" : $type="application/inc";break;
    case "html" : $type="application/html";break;
    case "pdf" : $type="application/pdf";break;
    case "exe" : $type="application/octet-stream";break;
    case "zip" : $type="application/zip";break;
    case "doc" : $type="application/msword";break;
    case "xls" : $type="application/vnd.ms-excel";break;
    case "ppt" : $type="application/vnd.ms-powerpoint";break;
    case "gif" : $type="image/gif";break;
    case "png" : $type="image/png";break;
    case "jpg" : $type="image/jpeg";break;
    default : $type="application/force-download";
}
```

```

header("Pragma: public");
header("Expires: 0");
header("Cache-Control: must-revalidate, post-check=0, pre-check=0");
header("Content-Type: $type");
$user_agent = strtolower ($_SERVER["HTTP_USER_AGENT"]);
if ((is_integer (strpos($user_agent, "msie")))
    && (is_integer (strpos($user_agent, "win"))))
{
    header("Content-Disposition: filename=$fichier;" );
}
else
{
    header("Content-Disposition: attachment; filename=$fichier;" );
}
header("Content-Transfer-Encoding: binary");
header("Content-Length: ".filesize($fichier));
readfile("$fichier");
exit();

```

12 Gérer les sockets

Il est possible de gérer le réseau de manière standard. La fonction *resource fsockopen (string target, int port [, int &errno [, string &errstr [, float timeout]]]*) initialise une connexion par socket à la ressource spécifiée par *target* sur le port spécifié par *port*. PHP supporte les cibles dans les domaines Internet et Unix. Cette fonction retourne un pointeur de fichier qui peut être utilisé avec d'autres fonctions fichiers, telles *fread*, *fwrite*, *fgets*, *fgetss*, *fputs*, *fclose* et *feof*. Si l'appel échoue, *fsockopen()* retourne FALSE, et si les options *errno* et *errstr* ont été fournies, elles contiennent désormais les raisons de l'échec. Suivant les environnements, le type 'domaine Unix' ou l'option *timeout* ne sont pas toujours disponibles.

Il existe d'autres fonctions réseau : *checkdnsrr*, *dns_get_record*, *gethostbyaddr*, *gethostbyname*, *gethostbyname_l*, *getprotobyname*, *getprotobyname_l*, *getservbyname*, *getservbyname_l*, *inet_ntop*, *inet_pton*, *pfsockopen*.

PHP peut aussi gérer les sockets avec une extension PHP. Cette extension, qui implémente une interface bas niveau, fournit la possibilité de fonctionner aussi bien sous forme de client que de serveur. Dans cette extension, de nombreuses fonctions ont le même nom que leur alter ego en langage C mais elles ont souvent des déclarations différentes.

La fonction *resource socket_create(int domain, int type, int protocol)* crée un point de communication (une socket) et retourne une ressource de socket. Le paramètre *domain* sélectionne le domaine à utiliser pour les communications. Actuellement, AF_INET (internet) et AF_UNIX (locales) sont les valeurs possibles de ce paramètre. Le paramètre *type* sélectionne le type de socket une des constantes suivantes : SOCK_STREAM, SOCK_DGRAM, SOCK_SEQPACKET, SOCK_RAW, SOCK_RDM ou SOCK_PACKET. Le paramètre *protocol* sélectionne le protocole qui peut être soit SOL_UDP, soit SOL_TCP.

La fonction *bool socket_connect(resource socket, string address [, int port])* crée une nouvelle connexion en utilisant la ressource *socket* (retournée par la fonction *socket_create*). Le paramètre *address* peut être soit une adresse IP (famille AF_INET) ou bien le chemin d'une socket (famille AF_UNIX). Le paramètre *port* n'est utilisé que lors de la connexion à

une socket de type AF_INET, et désigne le port sur l'hôte distant, sur lequel une connexion doit être faite.

La fonction *bool socket_bind(resource socket, string address [, int port])* lie le nom donné par *address* à la socket décrite par *socket*. Le paramètre d'adresse *address* peut être soit une adresse IP (famille AF_INET) ou bien le chemin d'une socket de domaine Unix (famille AF_UNIX). Le paramètre *port* n'est utilisé que lors de la connexion à une socket de type AF_INET, et désigne le port sur l'hôte distant, sur lequel une connexion doit être faite.

La fonction *bool socket_listen(resource socket [, int backlog])* permet à la socket *socket* d'être mise en attente de connexion entrantes. Un nombre maximum de *backlog* connexions seront mises en attente de traitement. Cette fonction n'est utilisable qu'avec des sockets de type SOCK_STREAM et SOCK_SEQPACKET.

La fonction *resource socket_accept(resource socket)* permet, lorsque la socket est en attente de connexion d'accepter des connexions. Une fois qu'une connexion est faite, une nouvelle ressource de socket est retournée.

La fonction *string socket_read(resource socket, int length [, int type])* lit des données dans la socket *socket*. Elle lit un maximum de *length* octets. Il est aussi possible d'utiliser \r, \n ou \0 pour terminer la lecture et ceci suivant la valeur choisie pour *type* :

- PHP_BINARY_READ : elle utilise la fonction système read (valeur par défaut en PHP >= 4.1.0),
- PHP_NORMAL_READ : la lecture s'arrête aux \n et \r (valeur par défaut en PHP <= 4.0.6).

Cette fonction peut retourner une chaîne de longueur zéro, qui indique la fin de la communication (l'autre socket a fermé la communication).

La fonction *int socket_write(resource socket, string buffer [, int length])* écrit dans la socket *socket* les données contenues dans *buffer*. Le paramètre optionnel *length* peut spécifier explicitement la taille des données qui doivent être écrites. Il est parfaitement valide que cette fonction retourne 0. Ceci signifie qu'aucun octet n'a été écrit. Pour être capable de distinguer une erreur (retour false) de ce cas, il faut utiliser l'opérateur ==.

La fonction *void socket_close(resource socket)* ferme la socket *socket*, et détruit toutes les ressources qui y sont liées.

Il existe d'autres fonctions : socket_create_listen, socket_getsockname, socket_recv, socket_recvfrom, socket_recvmsg, socket_select, socket_send, socket_sendmsg, socket_sendto, socket_shutdown ...